

Skills 101: Build Your Own Scientific Agent Skill

K-Dense Webinar · Skills 101 · Host: Yuhuan He

Wed, Jul 8, 2026, 1:30 PM – 3:00 PM PT

What you'll leave with: your own working Agent Skill, installed in Claude Code, plus a simple before/after showing it actually changed what the AI produced.

1. Intro — what is an Agent Skill, and why care?

An **Agent Skill** is a small folder that teaches a coding agent how to do something it isn't reliably good at on its own. It follows the open [Agent Skills](#) standard, so the same skill works across tools.

The one idea for today: a skill is **expertise you hand the model** — a workflow, a house style, the quirks of a niche package, a domain procedure. The model is already smart; a skill adds the specific knowledge and steps it's missing, and the skill's **description** is what makes the agent reach for it at the right moment.

A real use case

To see why this matters, we asked Claude to write a short scientific report on a topic — first **without** any skill, then **with** K-Dense's `scientific-writing` skill (plus its companions `research-lookup`, `scientific-schematics`, `generate-image`). Same model, same prompt:

- **Without a skill:** ~2 thin pages of text, no real figures, few or no sources.
- **With the skill:** a structured, properly-cited IMRAD report **with AI-generated figures** saved into a `figures/` folder.

That gap is the entire value of a skill. (We'll show the prepared before/after outputs during the intro; you don't need to run this one yourself.)

The anatomy of a skill

A skill is a folder. The only required file is `SKILL.md`:

```
my-skill/
├─ SKILL.md          # required: frontmatter + instructions
├─ references/       # optional: detail Claude reads only when needed
├─ scripts/          # optional: helper code Claude runs
└─ assets/           # optional: templates, examples, data
```

`SKILL.md` has two parts — **frontmatter** (a `name` and a load-bearing `description`) and a **body** (the instructions). We'll look at a real one together in the build, and the host will cover what makes a skill good (a description that triggers, teaching only what the model doesn't know, and keeping it concise).

2. Environment setup & pre-reqs

You will need **Claude Code** + **an editor you like**.

What to have ready

1. **Install Claude Code** and sign in with a **Claude Pro or Max plan**:

- Install / docs: <https://code.claude.com/docs/en/overview>
- Skills docs (good to skim): <https://code.claude.com/docs/en/skills>
- If `claude` isn't installed yet (needs Node.js 18+): `npm install -g @anthropic-ai/claude-code`

2. **An IDE you like (e.g. Visual Studio Code)**: After installing Claude Code, run it **inside your IDE's integrated terminal**. That way one window lets you **edit files**, **run `claude`**, and **watch the file tree change**.

Clone the workshop repo

Everything for today lives in one GitHub repo. Clone it, open it in your IDE, and run `claude` from the repo root:

```
git clone https://github.com/K-Dense-AI/skill-101-webinar.git
cd skill-101-webinar
```

Open this folder in **VS Code** or **Cursor** (File → Open Folder). You'll edit skills in the sidebar and run `claude` in the integrated terminal — one window.

Set up Claude Code & create your skills folder

We'll create the skills folder **inside this repo** during the session (the repo doesn't ship with it pre-made). From the repo root:

```

claude --version                # confirms Claude Code is installed
mkdir -p .claude/skills .claude/skills-disabled # project skills live here
ls .claude/skills                # empty until you install a skill – that's
fine
claude                          # launch from the repo root

```

Skills you install today stay in **this project only** — they won't touch your global `~/.claude/skills/` folder. If Claude asks you to **trust this workspace**, accept so project skills can load.

Repo layout

After cloning, you should see:

```

skill-101-webinar/
├── .claude/
│   ├── skills/                ← you create this; warm-up + your skills go here
│   └── skills-disabled/       ← move skills here to turn them OFF
└── workshop-pack/
    ├── example_skill/plain-language-summary/ ← warm-up skill
    ├── sample_inputs/          ← text to paste while testing
    │   ├── kdense_blog.txt
    │   ├── messy_meeting_notes.txt
    │   ├── abstract.txt
    │   ├── experiment_notes.txt
    │   └── results.csv
    ├── templates/
    │   ├── your-skill-template.md ← blank skill to copy for your own
    │   └── paper-draft-SKILL.md   ← full paper-draft skill (open build)
    └── use_case/                 ← prepared before/after for the intro
        ├── CRISPR_no_skill.pdf   ← no-skill report (thin, no figures)
        └── CRISPR_with_skill.pdf ← with scientific-writing (has figures)

```

Quick check (run from the repo root):

```

ls workshop-pack # example_skill sample_inputs templates use_case
ls .claude/skills # empty until the warm-up

```

All sample inputs are **synthetic or public** — safe to paste. You'll use `workshop-pack/sample_inputs/kdense_blog.txt` in the warm-up, `workshop-pack/sample_inputs/abstract.txt` and `workshop-pack/templates/your-skill-template.md` in the open build; the others are optional. The two PDFs in `workshop-`

`pack/use_case/` are the **prepared before/after reports** for the intro — you don't run that demo yourself, but you can open them to see the difference.

3. The goal

Clear objective (everyone):

Understand how to use a skill, then **build your own** Agent Skill from scratch, install it in Claude Code, and show the **before/after** — the same prompt producing a clearly better result with your skill on.

Reach objective (go further):

Build a **more complex skill** that uses a `references/` file (and optionally a `scripts/` helper or `assets/` template), and write a tiny **eval** — score the output with vs. without your skill on a short rubric you design.

4. Warm-up — install a skill and feel the difference

We'll install a small, ready-made writing skill and watch it change Claude's output.

a) See what Claude does on its own (no skill). Launch `claude` from the repo root, paste in the sample blog post (`workshop-pack/sample_inputs/kdense_blog.txt`), and ask:

Give me a plain-language summary of this.

Notice the shape of the answer — it's fine, but free-form.

b) Install the example skill (from the workshop pack). Copy the example skill folder into your project skills folder:

```
cp -R workshop-pack/example_skill/plain-language-summary .claude/skills/
```

Then, **restart Claude Code** (skills load at startup). You should see `.claude/skills/plain-language-summary/SKILL.md` in your IDE sidebar.

c) Run the same prompt again. Now the answer follows the skill's fixed structure.

Turning a skill OFF (to compare). Renaming a skill in place does **not** disable it — you must move it **out** of `.claude/skills/`. Toggle with `mv`, restarting Claude each time:

```
mv .claude/skills/plain-language-summary .claude/skills-disabled/ # OFF, then restart
claude
mv .claude/skills-disabled/plain-language-summary .claude/skills/ # ON, then restart
claude
```

5. Open build: create your own skill

We'll build one complete skill **together** — a **paper-draft** skill that expands a paper abstract into a structured IMRAD draft (Introduction, Methods, Results, Discussion). Follow these steps; then in your group, **swap in your own idea** and repeat.

Step 0 : Baseline first

Before building anything, in **claude** paste the sample abstract (**workshop-pack/sample_inputs/abstract.txt**) and ask — keep the output:

Expand this abstract into a full paper draft with Introduction, Methods, Results, and Discussion: "<paste the text of workshop-pack/sample_inputs/abstract.txt here>"

Notice what the base model does — expanding an abstract means adding detail that isn't there, and the base model tends to **invent plausible-sounding citations and specific numbers** to fill the gaps.

Step 1 : Make the folder

The folder name is the skill name: lowercase, hyphens only.

```
mkdir -p .claude/skills/paper-draft
```

We'll build the skill directly in **.claude/skills/** so it shows up in your IDE sidebar and Claude can load it from the project.

Step 2 : **SKILL.md** with a load-bearing description

Create a file called **SKILL.md** inside the **paper-draft** folder you just made (so it ends up at **.claude/skills/paper-draft/SKILL.md**):

```
touch .claude/skills/paper-draft/SKILL.md # creates the file; then open it in your
IDE
```

(In an IDE you can skip the command: expand `.claude/skills/paper-draft/` in the file tree → **New File** → name it `SKILL.md`.)

Open it and paste the frontmatter:

```
---
name: paper-draft
description: Expand a paper abstract or short study summary into a structured IMRAD
draft – Introduction, Methods, Results, Discussion – written in full paragraphs,
staying strictly grounded in the abstract and inserting placeholders for anything that
needs a real source or measurement instead of inventing it. Use when the user pastes an
abstract or asks to expand, draft, flesh out, or write a paper, manuscript, or report
from an abstract or summary.
---
```

- `name` must match the folder name (`paper-draft`).
- The `description` is **load-bearing** — write it in the third person and pack in the words a user would actually type ("expand this abstract", "draft a paper", "flesh out a manuscript").

Step 3 : Write the body: the exact output you want

Below the closing `---`, add the body — four sections: **When to use**, **Output format** (the IMRAD headers + a closing *Gaps to fill* list), **Rules** (the misses to prevent — e.g. never invent citations or numbers; insert `[citation needed]` / `[TODO]` instead; keep every number from the abstract exact), and one worked **Example**. The host will walk through it live; the full text to paste is in the pack at `workshop-pack/templates/paper-draft-SKILL.md`.

Step 4 : Load it into Claude Code

Your skill is already in `.claude/skills/paper-draft/`. **Restart** `claude` (from the repo root) so Claude picks up the new file.

Step 5 : Test that it triggers

Paste the **same abstract from Step 0** (without naming the skill) and check the reply uses the IMRAD structure, keeps the abstract's exact numbers, and — crucially — uses `[citation needed]` / `[TODO]` placeholders plus a **Gaps to fill** list instead of inventing sources.

If it doesn't fire automatically: run it explicitly with `/paper-draft`, then paste the abstract. If that works, your skill is good and the **description** just needs more trigger words.

Step 6 : Prove it helped

1. **With skill ON** — run the Step 0 abstract, keep the output.
2. **Turn the skill OFF** — move it out and restart:

```
mv .claude/skills/paper-draft .claude/skills-disabled/
```

Run the **same** prompt, keep that output. 3. **Turn it back ON:**

```
mv .claude/skills-disabled/paper-draft .claude/skills/
```

4. **Score them** on a tiny rubric you design — fill it in from your own two outputs:

Check	Without skill	With skill
Used the IMRAD structure?		
Added no numbers or results beyond the abstract?		
Invented no citations (used <code>[citation needed]</code> / <code>[TODO]</code> instead)?		
Listed the gaps to fill at the end?		

Now do your own

Repeat Steps 1–6 with **your** idea. Copy the blank template from the workshop pack (`workshop-pack/templates/your-skill-template.md`). Good first skill = **a format or workflow with a clear "right answer"** so the before/after is obvious, and **no API keys needed**.

5B. Reach: references, scripts & assets (optional)

A simple skill is just one `SKILL.md`. As skills get bigger, they use three optional folders. The difference is **what the agent does with each**:

Folder	The agent...	What it holds
<code>references/</code>	reads it (for knowledge)	Markdown docs, loaded only when needed
<code>scripts/</code>	runs it (executes code)	helper scripts for fragile/exact steps (e.g., call an API, parse a file)
<code>assets/</code>	uses/copies it (materials)	templates, styles, sample data

Example: for `paper-draft`, add a `references/imrad.md` with your lab's or target journal's exact section conventions (e.g. word limits, what belongs in Methods vs. Results, a house

citation style), then link it from `SKILL.md` with `For section conventions, see references/imrad.md`. Claude reads it only when needed.

6. Demos

Each group, 5–8 minutes (no slides needed):

1. **Your skill** — what it does and who it's for
 2. **The before/after** — the same prompt with vs. without it (screen-share both outputs).
 3. **One surprise** — something you learned (a description tweak that fixed triggering, a failure, an idea it unlocked).
-

7. Links

- Claude Code (install & docs): <https://code.claude.com/docs/en/overview>
 - Claude Code **Skills** docs: <https://code.claude.com/docs/en/skills>
 - Agent Skills standard: <https://agentskills.io/>
 - Anthropic skill-authoring guide: <https://claude.com/docs/skills/how-to>
 - Example science skills (inspiration): <https://github.com/K-Dense-AI/scientific-agent-skills>
 - Workshop repo: <https://github.com/K-Dense-AI/skill-101-webinar>
-

8. Extra resources

Writing a description that actually triggers

- Third person: "Generates X...", not "I can help you...".
- Include **what** + **when** + the **words users type**.
- Be specific: "Use when working with PDFs, forms, or document extraction" beats "helps with documents".

Make your skill earn its place

- Target **niche knowledge or a repeatable workflow**, not common-knowledge tasks.
- Keep `SKILL.md` tight; move depth into `references/`.
- Prefer a **script** for fragile, must-be-consistent steps.

Troubleshooting

- **Skill never fires** → fix the description (add trigger words); or call it explicitly with `/skill-name`.
- **Claude doesn't see it** → restart Claude Code; confirm the folder is `.claude/skills/<name>/SKILL.md` and `name` matches the folder; accept the workspace trust dialog if prompted.
- **Output didn't change** → your `SKILL.md` may be too vague; add an explicit output format + one example.
- **Can't turn a skill off** → moving/renaming it *inside* `.claude/skills/` doesn't disable it; move it **out** to `.claude/skills-disabled/` and restart.
- **Extra skills show up in `/skills`** → you may have skills in your global `~/.claude/skills/` from past experiments; ignore them for today's exercises.

Skill ideas if you're stuck

Plain-language paper summary · methods-paragraph writer · lab-protocol checklist · weekly research update · a figure/plot house-style · a niche-package quickstart · meeting-notes formatter · commit-message style · code-review checklist.